# 记忆系统：让Agent拥有经验

- 来源: https://class.imooc.com/lesson/5290#mid=105151
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: Agent的核心架构剖析
- 课节: 图文 1-3 记忆系统：让Agent拥有经验

## 1-3 记忆系统：让Agent拥有经验

## 记忆系统：让Agent拥有经验

> [!TIP] 阅读指南 想象一下，你每天醒来都忘记了昨天发生的一切。你不记得朋友的名字，不记得学过的知识，甚至不记得自己是谁。这样的生活会是什么样? 没有记忆的 Agent 就是这样。每次对话都是“初次见面”,每个任务都从零开始。 记忆系统让 Agent 能够: 积累经验: 从过去的交互中学习 保持连贯: 理解上下文和历史 个性化服务: 记住用户的偏好和习惯 持续进化: 随着时间推移变得更智能 本节我们将深入剖析 Agent 的记忆系统——这个让 Agent 从“工具”升级为“助手”的关键能力。

## 回顾：Agent 的记忆系统

在第3节我们了解了 Agent 架构中的记忆系统——它让 Agent 能够"记住"过去的交互。现在深入探讨：如何实现一个完整的记忆系统？在本节，我们来看看如何在代码层面实现这些记忆，它们如何协同工作?

## 四种记忆的工作原理

### 短期记忆(Short-term Memory)

核心作用: 维持当前对话的上下文连贯性。

核心作用

工作原理:

工作原理

- 将对话历史直接保存在内存列表中
- 每次向 LLM 发送请求时，携带完整的对话历史
- 当超出 token 限制时，采用滑动窗口删除最早的消息

典型场景:

典型场景

```
用户:"帮我订外卖"
Agent:"好的,想吃什么?"
用户:"川菜"  ← 这里 Agent 要记住"订外卖"这个上下文
Agent:"推荐麻辣香锅..."
```

技术要点:

技术要点

- 存储方式:Python 列表或 LangChain 的 ConversationBufferMemory
- 容量限制:通常保留最近 10-20 轮对话
- 生命周期:会话结束即清空

> 上节，我们实现的Log Agent案例就是使用的这种短期记忆。

## 工作记忆(Working Memory)

核心作用: 跟踪多步任务的执行状态。

- 使用状态字典记录任务的关键变量
- 类似函数的局部变量，任务结束即释放
- 在 LangGraph 中对应 State 对象

```
任务:订外卖

工作记忆:
  当前步骤 → "选择餐厅"
  已完成步骤 → ["确认菜系", "筛选餐厅"]
  待办步骤 → ["确认地址", "支付"]
  临时数据 → {餐厅:"川味轩", 总价:68元}
```

- 存储方式:字典结构或 Pydantic 模型
- 作用范围:单个任务流程
- 生命周期:任务完成后清空或归档到长期记忆

## 长期记忆(Long-term Memory)

核心作用: 跨会话保存用户偏好和历史知识。

- 将重要信息持久化到数据库或文件
- 使用向量检索技术根据语义相关性召回
- 只检索与当前问题最相关的 top-k 条记忆

```
第一天:
用户:"我喜欢吃辣的"
→ 保存到长期记忆:偏好=重口味

三天后:
用户:"给我推荐个餐厅"
→ 从长期记忆检索到"偏好=重口味"
Agent:"推荐一家川菜馆,他们的麻辣香锅很正宗"
```

- 存储方式: 结构化数据 → SQLite/PostgreSQL 非结构化知识 → 向量数据库(Chroma、Pinecone)
- 检索策略:基于向量相似度的 top-k 检索
- 生命周期:永久保存(可设置遗忘策略)

## 情景记忆(Episodic Memory)

核心作用: 记录完整的交互事件，用于分析和审计。

- 类似"飞机的黑匣子",记录每次交互的完整细节

- 以时间序列的方式存储
- 主要用于事后分析，不直接参与推理
- 存储方式:日志文件或时序数据库
- 用途:问题排查、用户行为分析、模型优化
- 生命周期:长期归档

### 记忆类型对比

| 记忆类型 | 生命周期 | 容量 | 访问速度 | 主要用途 |
| --- | --- | --- | --- | --- |
| 短期记忆 | 单次会话 | 小 | 极快 | 对话上下文 |
| 工作记忆 | 任务期间 | 中 | 快 | 任务状态 |
| 长期记忆 | 永久 | 大 | 较慢 | 知识积累 |
| 情景记忆 | 永久 | 很大 | 慢 | 审计分析 |

## 记忆的检索与管理策略

### 分层检索：按需加载

核心思想: 不是所有记忆都要塞进 Prompt,而是根据相关性智能召回。

核心思想

> 术语说明:召回(Recall) "召回"是信息检索领域的专业术语，指从大量数据中找出相关内容的过程。你可以把它理解为: 图书馆场景: 从整个图书馆的百万藏书中，找出与你研究主题相关的10本书 记忆系统: 从Agent的海量长期记忆中，检索出与当前问题最相关的几条记忆 召回的关键指标: 召回率: 找出了多少相关内容(越高越全面) 准确率: 找出的内容有多少真正相关(越高越精准) 在RAG和Agent系统中，"召回"通常指: 用户提出问题 系统从向量数据库中检索相关文档/记忆 返回top-k条最相关的结果 这个过程就叫"召回",它决定了后续LLM能看到哪些背景知识。

检索策略对比:

检索策略对比

| 记忆类型 | 检索方式 | 原因 |
| --- | --- | --- |
| 短期记忆 | 全量返回 | 本身就很小 |
| 工作记忆 | 任务相关部分 | 只返回当前任务 |
| 长期记忆 | Top-k 检索 | 太大，按相关性筛选 |
| 情景记忆 | 按需查询 | 主要用于分析 |

混合检索流程:

混合检索流程

```
用户问题:"给我推荐个餐厅"

↓
1. 短期记忆 → 读取最近 10 轮对话
2. 工作记忆 → 检查是否有进行中的任务
3. 长期记忆 → 向量检索相关偏好(top-5)
   - "用户喜欢川菜"(相似度 0.89)
   - "不吃海鲜"(相似度 0.76)
   - "预算 50-100"(相似度 0.72)
↓
组合成完整上下文 → 发送给 LLM
```

## 记忆更新：从对话中学习

核心机制: Agent 需要判断哪些信息值得保存到长期记忆。

核心机制

更新流程:

更新流程

```
用户:"我不喜欢吃香菜"

↓
Agent 分析:这是一个用户偏好!

↓
提取结构化信息:
  类型:饮食偏好
  内容:不喜欢香菜

↓
保存到长期记忆:
  user_preferences.food_dislikes = ["香菜"]
```

实现方式:

实现方式

- 规则提取: 用正则匹配"我喜欢。...", “我不要。...” （最传统的方式）

规则提取

- LLM 提取: 让 LLM 分析对话，返回结构化偏好

LLM 提取

- 主动确认: “您需要我记录您不吃香菜的偏好吗?”

主动确认

## 记忆遗忘：避免无限膨胀

遗忘的必要性

- 记忆越多，检索越慢
- 过时的信息会干扰决策
- 隐私保护要求定期清理

遗忘的三种处理方式

"遗忘"不等于直接删除，它包含三种处理方式:

| 处理方式 | 说明 |
| --- | --- |
| 删除 | 彻底移除记忆(不可恢复) |
| 压缩 | 保留摘要，删除细节(节省空间) |
| 降级 | 从热存储移到冷存储(降低成本) |

遗忘策略

策略 1: 基于时间

判断标准：创建时间超过 90 天

处理方式：

- 重要记忆 → 保留
- 普通记忆 → 压缩(保留摘要)
- 临时记忆 → 删除

策略 2: 基于访问频次

判断标准：近 30 天访问次数

- 高频访问(>10次) → 保留原文
- 低频访问(1-10次) → 压缩
- 从未访问(0次) → 删除

策略 3: 基于重要性

判断标准：记忆的重要性标记

- 用户明确设置的偏好 → 永久保留
- 系统自动提取的信息 → 定期压缩
- 临时会话数据 → 定期删除

遗忘实现示例:

遗忘实现示例

```
定期任务(每周执行):

1. 扫描长期记忆
   → 遍历所有记忆条目

2. 应用遗忘策略:

   删除操作:
   - 创建时间 > 90 天 且 访问次数 = 0 且 重要性 = 低
   → 直接删除

   压缩操作:
   - 创建时间 > 30 天 且 访问次数 < 3 且 重要性 = 中
   → 保留摘要,删除原文
   示例:
     原文:"用户在2024-10-01订了川菜,选了宫保鸡丁...(200字)"
     压缩:"用户偏好川菜"

   降级操作:
   - 创建时间 > 7 天 且 访问频次下降
   → 从向量数据库移到普通数据库

3. 压缩情景记忆日志:
   - 合并相似事件
   - 删除冗余字段
   - 归档到冷存储
```

> 为什么需要压缩而非直接删除? 删除会丢失信息，而压缩可以: 保留核心要点(如"用户喜欢川菜") 删除冗余细节(如具体菜品、时间) 节省存储空间(原文200字 → 摘要10字) 保持历史连贯性(不会出现记忆断层) 类似人类记忆:你可能不记得3个月前某天吃了什么菜，但记得自己喜欢川菜。

## 完整案例：游戏助手的记忆协同

通过一个游戏助手的例子，看四种记忆如何协同工作。

场景1：初次对话(短期+长期记忆)

```
═══════════════════════════════════════════════════════
用户: "我是新手,选什么职业好?"
═══════════════════════════════════════════════════════
[短期记忆] 记录本轮对话
[长期记忆] 查询:用户之前选过职业吗?
结果:无记录
═══════════════════════════════════════════════════════
Agent: "你可以选战士、法师或弓箭手,
你喜欢近战还是远程?"
═══════════════════════════════════════════════════════


═══════════════════════════════════════════════════════
用户: "我喜欢近战"
═══════════════════════════════════════════════════════
[短期记忆] 记录:"喜欢近战"
[长期记忆] 保存:偏好=近战/战士
═══════════════════════════════════════════════════════
Agent: "推荐战士!血量高、防御强..."
═══════════════════════════════════════════════════════
```

场景2：执行任务(工作记忆)

```
──────────────────────────────────────────────────────
用户: "帮我开始'击败史莱姆王'任务"
──────────────────────────────────────────────────────
[工作记忆] 初始化任务状态:
task_name = "击败史莱姆王"
current_step = 1
steps = ["收集核心", "找巢穴", "战斗"]
completed = []
──────────────────────────────────────────────────────
Agent: "任务开始!第一步:收集 10 个史莱姆
核心"
──────────────────────────────────────────────────────

... (用户完成几个步骤)

──────────────────────────────────────────────────────
用户: "我现在该干嘛?"
──────────────────────────────────────────────────────
[工作记忆] 读取当前任务状态:
current_step = 3
completed = ["收集核心", "找巢穴"]
pending = ["战斗"]
──────────────────────────────────────────────────────
Agent: "你已经完成了前 3 步,现在去挑战
史莱姆王吧!"
──────────────────────────────────────────────────────

──────────────────────────────────────────────────────
任务完成!
──────────────────────────────────────────────────────
[工作记忆] → [长期记忆]
任务详情归档:
- 任务名称:击败史莱姆王
- 完成时间:2024-12-08
- 耗时:25 分钟
[工作记忆] 清空
──────────────────────────────────────────────────────
```

### 场景3：几天后再次登录(长期记忆)

```
──────────────────────────────────────────────────
用户: "我又来了,推荐个任务吧"
──────────────────────────────────────────────────
[长期记忆] 检索相关历史:
- 偏好职业:战士
- 已完成任务:["击败史莱姆王"]
- 游戏等级:15
──────────────────────────────────────────────────
Agent: "欢迎回来!看到你是 15 级战士,
推荐'地下城探险'任务,奖励丰富,
适合你的职业和等级"
──────────────────────────────────────────────────
```

Agent 像一个真正的游戏伙伴:

- 记住你的偏好(职业选择)
- 知道你的进度(完成的任务)
- 提供个性化建议(基于历史推荐)

# 下一节预告

但你可能已经注意到，光有记忆还不够——Agent 在面对复杂问题时，需要像人类一样深入思考、推理和规划。如何让 Agent 具备这种「慢思考」的能力?接下来，我们将深入探讨 Agent 的推理模式，揭示从 Chain of Thought 到 Tree of Thoughts 的演进路径，看看如何让 Agent 真正「会思考」。

# ■ 学点英语

| 中文 | English | 音标 | 说明 |
| --- | --- | --- | --- |
| 短期记忆 | Short-term Memory | /ʃɔːrt tɜːrm ˈmeməri/ | 维持当前对话上下文连贯性的临时记忆 |
| 工作记忆 | Working Memory | /ˈwɜːrkɪŋ ˈmeməri/ | 跟踪多步任务执行状态的过程记忆 |
| 长期记忆 | Long-term Memory | /lɔːŋ tɜːrm ˈmeməri/ | 跨会话持久化存储的用户偏好和历史知识 |
| 情景记忆 | Episodic Memory | /ˌepɪˈsɒdɪk ˈmeməri/ | 记录完整交互事件用于审计分析的记忆 |
| 召回 | Recall | /rɪˈkɔːl/ | 从大量数据中找出相关内容的过程 |
| 前K检索 | Top-k Retrieval | /tɒp keɪ rɪˈtriːvl/ | 基于相似度得分返回最相关的K条记忆的策略 |